DBMS Practical Exam

Complete Solutions — All 12 Problem Statements

Covers: SQL DDL · SQL DML · Joins · Subqueries · PL/SQL Blocks · Cursors · Stored Procedures · Triggers · MongoDB CRUD · Aggregation & Indexing

Problem Statement 01 — SQL DDL Statements

Q1. Create all four tables with appropriate data types and constraints.

```
-- EMPLOYEE
CREATE TABLE Employee (
employee_name VARCHAR2(50) PRIMARY KEY,
street VARCHAR2(100) NOT NULL,
city VARCHAR2(50) NOT NULL
);

-- COMPANY
CREATE TABLE Company (
company_name VARCHAR2(50) PRIMARY KEY,
city VARCHAR2(50) NOT NULL
);

-- WORKS
CREATE TABLE Works (
employee_name VARCHAR2(50) NOT NULL,
company_name VARCHAR2(50) NOT NULL,
salary NUMBER(10,2) NOT NULL,
PRIMARY KEY (employee_name, company_name),
FOREIGN KEY (employee_name) REFERENCES Employee(employee_name),
FOREIGN KEY (company_name) REFERENCES Company(company_name)
);

-- MANAGES
CREATE TABLE Manages (
employee_name VARCHAR2(50) NOT NULL,
manager_name VARCHAR2(50) NOT NULL,
PRIMARY KEY (employee_name),
FOREIGN KEY (employee_name) REFERENCES Employee(employee_name),
FOREIGN KEY (manager_name) REFERENCES Employee(employee_name)
);
```

Q2. Create a new table Department using Employee table (existing).

```
CREATE TABLE Department AS
SELECT employee_name, city FROM Employee;
```

Q3. Create Emp_Backup with same structure as Employee but no data.

```
CREATE TABLE Emp_Backup AS
SELECT * FROM Employee WHERE 1 = 0;
```

Q4. Add phone_number column to Employee table, then verify.

```
ALTER TABLE Employee ADD phone_number VARCHAR2(15);

-- Verify
```

```
DESCRIBE Employee;
```

Q5. Increase the size of the city column in Company table.

```
ALTER TABLE Company MODIFY city VARCHAR2(100);
```

Q6. Drop the phone_number column from Employee table.

```
ALTER TABLE Employee DROP COLUMN phone_number;
```

Q7. Rename the column salary to monthly_salary in Works table.

```
ALTER TABLE Works RENAME COLUMN salary TO monthly_salary;
```

Q8. Insert sample rows into Company, then remove all rows without deleting structure.

```
INSERT INTO Company VALUES ('First Bank Corporation', 'New York');
INSERT INTO Company VALUES ('Small Bank Corporation', 'Chicago');
INSERT INTO Company VALUES ('Tech Corp', 'San Francisco');

-- Remove all rows but keep structure
TRUNCATE TABLE Company;
```

Q9. Change salary type to DECIMAL(10,2) in Works table.

```
ALTER TABLE Works MODIFY salary DECIMAL(10,2);
```

Problem Statement 02 — SQL DML Statements

Schema Setup

```
CREATE TABLE Department (
Deptno NUMBER(2) PRIMARY KEY,
Dname VARCHAR2(20) NOT NULL,
Location VARCHAR2(20) NOT NULL
);

INSERT INTO Department VALUES (10, 'Accounting', 'Mumbai');
INSERT INTO Department VALUES (20, 'Research', 'Pune');
INSERT INTO Department VALUES (30, 'Sales', 'Nashik');
INSERT INTO Department VALUES (40, 'Operations', 'Nagpur');

CREATE TABLE Employee (
Empno NUMBER(4) PRIMARY KEY,
Ename VARCHAR2(30) NOT NULL,
Job VARCHAR2(15),
Mgr NUMBER(4),
Joined_date DATE,
Salary NUMBER(10,2),
Commission NUMBER(10,2),
Deptno NUMBER(2),
Address VARCHAR2(50),
FOREIGN KEY (Deptno) REFERENCES Department(Deptno)
);

INSERT INTO Employee VALUES (1001,'Nilesh Joshi', 'Clerk',
1005,TO_DATE('17-DEC-95','DD-MON-YY'),2800, 600, 20,'Nashik');
INSERT INTO Employee VALUES (1002,'Avinash Pawar', 'Salesman',
1003,TO_DATE('20-FEB-96','DD-MON-YY'),5000,1200, 30,'Nagpur');
INSERT INTO Employee VALUES (1003,'Amit Kumar', 'Manager',
1004,TO_DATE('02-APR-86','DD-MON-YY'),2000,NULL, 30,'Pune');
INSERT INTO Employee VALUES (1004,'Nitin Kulkarni', 'President',NULL,
TO_DATE('19-APR-86','DD-MON-YY'),50000,NULL,10,'Mumbai');
INSERT INTO Employee VALUES (1005,'Niraj Sharma', 'Analyst',
1003,TO_DATE('03-DEC-98','DD-MON-YY'),12000,NULL,20,'Satara');
INSERT INTO Employee VALUES (1006,'Pushkar
Deshpande','Salesman',1003,TO_DATE('01-SEP-96','DD-MON-YY'),6500,1500,30,'Pune');
INSERT INTO Employee VALUES (1007,'Sumit Patil', 'Manager',
1004,TO_DATE('01-MAY-91','DD-MON-YY'),25000,NULL,20,'Mumbai');
INSERT INTO Employee VALUES (1008,'Ravi Sawant', 'Analyst',
1007,TO_DATE('17-NOV-95','DD-MON-YY'),10000,NULL,NULL,'Amaravati');
```

Q1. Display employee information with explicit column names.

```
SELECT Empno AS "Employee Number",
```

```
Ename AS "Employee Name",
Job AS "Job Title",
Mgr AS "Manager ID",
Joined_date AS "Date of Joining",
Salary AS "Salary",
Commission AS "Commission",
Deptno AS "Department No",
Address AS "Address"
FROM Employee;
```

Q2. Display unique jobs from the table.

```
SELECT DISTINCT Job FROM Employee;
```

Q3. Change location of dept 40 to Bangalore.

```
UPDATE Department SET Location = 'Banglore' WHERE Deptno = 40;
COMMIT;
```

Q4. Change the name of employee 1003 to Nikhil Gosavi.

```
UPDATE Employee SET Ename = 'Nikhil Gosavi' WHERE Empno = 1003;
COMMIT;
```

Q5. Delete Pushkar Deshpande from employee table.

```
DELETE FROM Employee WHERE Ename = 'Pushkar Deshpande';
COMMIT;
```

Q6. Display employees whose job is 'Manager' or 'Analyst' (using OR and IN).

```
-- Using OR
SELECT * FROM Employee WHERE Job = 'Manager' OR Job = 'Analyst';

-- Using IN
SELECT * FROM Employee WHERE Job IN ('Manager', 'Analyst');
```

Q7. Display employee name and dept number for employees in dept 10, 20, 30, 40.

```
SELECT Ename, Deptno
FROM Employee
WHERE Deptno IN (10, 20, 30, 40);
```

Q8. Find names and joined dates of employees whose names start with 'A'.

```
SELECT Ename, Joined_date FROM Employee WHERE Ename LIKE 'A%';
```

Q9. Find names of employees having 'i' as second letter.

```
SELECT Ename FROM Employee WHERE Ename LIKE '_i%';
```

Q10. Find dept number and max salary where max salary > 5000.

```
SELECT Deptno, MAX(Salary) AS Max_Salary
FROM Employee
GROUP BY Deptno
HAVING MAX(Salary) > 5000;
```

Problem Statement 03 — SQL DML Using Joins

Schema Setup (same Employee/Works/Company/Manages schema)

```
-- (Tables created as in PS 01. Insert sample data below.)
INSERT INTO Company VALUES ('First Bank Corporation', 'New York');
INSERT INTO Company VALUES ('Small Bank Corporation', 'Chicago');
INSERT INTO Employee VALUES ('Alice', '123 Main St', 'New York');
INSERT INTO Employee VALUES ('Bob', '456 Oak Ave', 'Chicago');
INSERT INTO Employee VALUES ('Carol', '789 Pine Rd', 'New York');
INSERT INTO Works VALUES ('Alice', 'First Bank Corporation', 12000);
INSERT INTO Works VALUES ('Bob', 'Small Bank Corporation', 8000);
INSERT INTO Works VALUES ('Carol', 'First Bank Corporation', 9000);
INSERT INTO Manages VALUES ('Alice', 'Carol');
```

Q1. Find names of employees who work for First Bank Corporation.

```
SELECT e.employee_name
FROM Employee e
JOIN Works w ON e.employee_name = w.employee_name
WHERE w.company_name = 'First Bank Corporation';
```

Q2. Find names and cities of all employees who work for First Bank Corporation.

```
SELECT e.employee_name, e.city
FROM Employee e
JOIN Works w ON e.employee_name = w.employee_name
WHERE w.company_name = 'First Bank Corporation';
```

Q3. Find names, streets, cities of employees at First Bank Corporation earning > $10000.

```
SELECT e.employee_name, e.street, e.city
FROM Employee e
JOIN Works w ON e.employee_name = w.employee_name
WHERE w.company_name = 'First Bank Corporation'
AND w.salary > 10000;
```

Q4. Find all employees who earn more than EACH employee of Small Bank Corporation.

```
SELECT DISTINCT w1.employee_name
FROM Works w1
JOIN Works w2 ON w2.company_name = 'Small Bank Corporation'
GROUP BY w1.employee_name
HAVING MIN(w1.salary) > MAX(w2.salary);
```

Q5. Find all employees who earn more than the average salary of their own company.

```
SELECT w.employee_name
FROM Works w
JOIN (
SELECT company_name, AVG(salary) AS avg_sal
FROM Works
GROUP BY company_name
) avg_w ON w.company_name = avg_w.company_name
```

```
WHERE w.salary > avg_w.avg_sal;
```

Q6. Find the company that has the smallest payroll.

```
SELECT company_name, SUM(salary) AS total_payroll
FROM Works
GROUP BY company_name
ORDER BY total_payroll ASC
FETCH FIRST 1 ROW ONLY;
```

Q7. Find companies whose avg salary is higher than avg salary at First Bank Corporation.

```
SELECT w.company_name, AVG(w.salary) AS avg_salary
FROM Works w
GROUP BY w.company_name
HAVING AVG(w.salary) > (
SELECT AVG(salary)
FROM Works
WHERE company_name = 'First Bank Corporation'
);
```

Q8. Give all employees of First Bank Corporation a 10% raise.

```
UPDATE Works
SET salary = salary * 1.10
WHERE company_name = 'First Bank Corporation';
COMMIT;
```

Q9. Insert names and salaries of employees earning more than average into HighEarners.

```
CREATE TABLE HighEarners AS
SELECT employee_name, salary
FROM Works
WHERE salary > (SELECT AVG(salary) FROM Works);
```

Q10. Delete employees who work for a company located in Gotham.

```
DELETE FROM Employee
WHERE employee_name IN (
SELECT w.employee_name
FROM Works w
JOIN Company c ON w.company_name = c.company_name
WHERE c.city = 'Gotham'
);
COMMIT;
```

Problem Statement 04 — SQL DML Using Subqueries

Uses the same Employee / Works / Company / Manages schema as PS 03.

Q1. Find names of employees who work for First Bank Corporation.

```
SELECT employee_name
FROM Works
WHERE company_name = 'First Bank Corporation';
```

Q2. Find names and cities of employees who work for First Bank Corporation.

```
SELECT employee_name, city
FROM Employee
WHERE employee_name IN (
SELECT employee_name
FROM Works
WHERE company_name = 'First Bank Corporation'
);
```

Q3. Find names, street, city of employees at First Bank Corp earning > $10000.

```
SELECT employee_name, street, city
FROM Employee
WHERE employee_name IN (
SELECT employee_name
FROM Works
WHERE company_name = 'First Bank Corporation'
AND salary > 10000
);
```

Q4. Find employees earning more than EVERY employee of Small Bank Corporation.

```
SELECT employee_name
FROM Works
WHERE salary > ALL (
SELECT salary
FROM Works
WHERE company_name = 'Small Bank Corporation'
);
```

Q5. Find employees earning more than the average salary of their company.

```
SELECT employee_name
FROM Works w1
WHERE salary > (
SELECT AVG(salary)
FROM Works w2
WHERE w2.company_name = w1.company_name
);
```

Q6. Find the company with the smallest payroll.

```
SELECT company_name
FROM Works
GROUP BY company_name
```

```
HAVING SUM(salary) = (
SELECT MIN(total_pay)
FROM (
SELECT SUM(salary) AS total_pay
FROM Works
GROUP BY company_name
)
);
```

Q7. Find companies whose avg salary is higher than avg at First Bank Corporation.

```
SELECT company_name
FROM Works
GROUP BY company_name
HAVING AVG(salary) > (
SELECT AVG(salary)
FROM Works
WHERE company_name = 'First Bank Corporation'
);
```

Q8. Give all First Bank Corporation employees a 10% raise.

```
UPDATE Works
SET salary = salary * 1.10
WHERE company_name = 'First Bank Corporation';
COMMIT;
```

Q9. Insert employees earning above average into HighEarners table.

```
INSERT INTO HighEarners (employee_name, salary)
SELECT employee_name, salary
FROM Works
WHERE salary > (SELECT AVG(salary) FROM Works);
COMMIT;
```

Q10. Delete employees working for a Gotham-based company.

```
DELETE FROM Employee
WHERE employee_name IN (
SELECT employee_name
FROM Works
WHERE company_name IN (
SELECT company_name
FROM Company
WHERE city = 'Gotham'
)
);
COMMIT;
```

Problem Statement 05 — PL/SQL Unnamed Block (Control Structures)

Schema Setup

```
CREATE TABLE Borrower (
Rollin NUMBER(10) PRIMARY KEY,
Name VARCHAR2(50) NOT NULL,
DateofIssue DATE NOT NULL,
NameofBook VARCHAR2(100) NOT NULL,
Status CHAR(1) DEFAULT 'I' CHECK (Status IN ('I','R'))
);

CREATE TABLE Fine (
Roll_no NUMBER(10),
Date DATE,
Amt NUMBER(10,2),
FOREIGN KEY (Roll_no) REFERENCES Borrower(Rollin)
);

-- Sample data
INSERT INTO Borrower VALUES (101, 'Rahul', TO_DATE('01-MAR-2025','DD-MON-YYYY'), 'DBMS',
'I');
INSERT INTO Borrower VALUES (102, 'Priya', TO_DATE('10-FEB-2025','DD-MON-YYYY'), 'OS',
'I');
COMMIT;
```

PL/SQL Block

```
DECLARE
v_roll Borrower.Rollin%TYPE;
v_book Borrower.NameofBook%TYPE;
v_issue_date Borrower.DateofIssue%TYPE;
v_status Borrower.Status%TYPE;
v_days NUMBER;
v_fine_amt NUMBER := 0;
BEGIN
-- Accept inputs from user (substitution variables)
v_roll := &roll_no;
v_book := '&book_name';

-- Fetch date of issue and status
SELECT DateofIssue, Status
INTO v_issue_date, v_status
FROM Borrower
WHERE Rollin = v_roll AND NameofBook = v_book;
```

```
-- Calculate number of days since issue
v_days := TRUNC(SYSDATE) - TRUNC(v_issue_date);

DBMS_OUTPUT.PUT_LINE('Days since issue: ' || v_days);

-- Calculate fine
IF v_days BETWEEN 15 AND 30 THEN
v_fine_amt := v_days * 5;
ELSIF v_days > 30 THEN
-- Rs.50/day for days > 30, Rs.5/day for first 30 days
v_fine_amt := (30 * 5) + ((v_days - 30) * 50);
ELSE
v_fine_amt := 0;
DBMS_OUTPUT.PUT_LINE('No fine applicable (within 15 days).');
END IF;

-- Update book status to 'R' (Returned)
UPDATE Borrower
SET Status = 'R'
WHERE Rollin = v_roll AND NameofBook = v_book;

-- Insert fine record if applicable
IF v_fine_amt > 0 THEN
INSERT INTO Fine VALUES (v_roll, SYSDATE, v_fine_amt);
DBMS_OUTPUT.PUT_LINE('Fine of Rs.' || v_fine_amt || ' recorded.');
END IF;

COMMIT;
DBMS_OUTPUT.PUT_LINE('Book returned successfully.');

EXCEPTION
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('No matching borrower/book record found.');
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Problem Statement 06 — PL/SQL Cursors

Schema Setup

```
CREATE TABLE Stud_Marks (
Roll NUMBER(10) PRIMARY KEY,
Name VARCHAR2(50) NOT NULL,
Total_Marks NUMBER(4) NOT NULL
);

CREATE TABLE Result (
Roll NUMBER(10) PRIMARY KEY,
Name VARCHAR2(50),
Class VARCHAR2(30)
);

-- Sample data
INSERT INTO Stud_Marks VALUES (1, 'Aarav', 1200);
INSERT INTO Stud_Marks VALUES (2, 'Meera', 950);
INSERT INTO Stud_Marks VALUES (3, 'Rohit', 860);
INSERT INTO Stud_Marks VALUES (4, 'Sanjana', 800);
COMMIT;
```

Stored Procedure with Explicit Cursor

```
CREATE OR REPLACE PROCEDURE proc_Grade AS
-- Explicit Cursor Declaration
CURSOR c_students IS
SELECT Roll, Name, Total_Marks FROM Stud_Marks;

v_roll Stud_Marks.Roll%TYPE;
v_name Stud_Marks.Name%TYPE;
v_marks Stud_Marks.Total_Marks%TYPE;
v_class Result.Class%TYPE;
BEGIN
OPEN c_students;
LOOP
FETCH c_students INTO v_roll, v_name, v_marks;
EXIT WHEN c_students%NOTFOUND;

-- Categorize
IF v_marks >= 990 AND v_marks <= 1500 THEN
v_class := 'Distinction';
ELSIF v_marks >= 900 AND v_marks <= 989 THEN
v_class := 'First Class';
ELSIF v_marks >= 825 AND v_marks <= 899 THEN
v_class := 'Higher Second Class';
```

```
ELSE
v_class := 'Pass';
END IF;

-- Insert into Result table
INSERT INTO Result VALUES (v_roll, v_name, v_class);
END LOOP;
CLOSE c_students;
COMMIT;
DBMS_OUTPUT.PUT_LINE('Grading complete.');
END proc_Grade;
/

-- Execute the procedure
BEGIN
proc_Grade;
END;
/

-- Verify results
SELECT * FROM Result;
```

Problem Statement 07 — Stored Procedure & Function

Uses same Stud_Marks and Result schema as PS 06.

Stored Procedure — proc_Grade

```
CREATE OR REPLACE PROCEDURE proc_Grade(
p_roll IN Stud_Marks.Roll%TYPE,
p_class OUT Result.Class%TYPE
) AS
v_marks Stud_Marks.Total_Marks%TYPE;
BEGIN
SELECT Total_Marks INTO v_marks
FROM Stud_Marks
WHERE Roll = p_roll;

IF v_marks >= 990 AND v_marks <= 1500 THEN
p_class := 'Distinction';
ELSIF v_marks >= 900 AND v_marks <= 989 THEN
p_class := 'First Class';
ELSIF v_marks >= 825 AND v_marks <= 899 THEN
p_class := 'Higher Second Class';
ELSE
p_class := 'Pass';
END IF;
EXCEPTION
WHEN NO_DATA_FOUND THEN
p_class := 'Student not found';
END proc_Grade;
/
```

Stored Function — fn_Grade

```
CREATE OR REPLACE FUNCTION fn_Grade(p_marks IN NUMBER)
RETURN VARCHAR2 AS
BEGIN
IF p_marks >= 990 AND p_marks <= 1500 THEN
RETURN 'Distinction';
ELSIF p_marks >= 900 AND p_marks <= 989 THEN
RETURN 'First Class';
ELSIF p_marks >= 825 AND p_marks <= 899 THEN
RETURN 'Higher Second Class';
ELSE
RETURN 'Pass';
END IF;
END fn_Grade;
/
```

Anonymous PL/SQL Block to Populate Result Table

```
DECLARE
v_class Result.Class%TYPE;
v_roll Stud_Marks.Roll%TYPE;
CURSOR c_stud IS SELECT Roll FROM Stud_Marks;
BEGIN
OPEN c_stud;
LOOP
FETCH c_stud INTO v_roll;
EXIT WHEN c_stud%NOTFOUND;

-- Call procedure
proc_Grade(v_roll, v_class);

-- Insert/Update Result
MERGE INTO Result r
USING (SELECT v_roll AS roll, v_class AS cls FROM DUAL) src
ON (r.Roll = src.roll)
WHEN MATCHED THEN
UPDATE SET r.Class = src.cls
WHEN NOT MATCHED THEN
INSERT (Roll, Name, Class)
SELECT v_roll, Name, v_class FROM Stud_Marks WHERE Roll = v_roll;
END LOOP;
CLOSE c_stud;
COMMIT;
DBMS_OUTPUT.PUT_LINE('Result table populated successfully.');
END;
/
```

Problem Statement 08 — Database Triggers

Schema Setup

```
CREATE TABLE Library (
Book_id NUMBER(10) PRIMARY KEY,
Book_name VARCHAR2(100) NOT NULL,
Author VARCHAR2(60),
Status VARCHAR2(20),
Issued_to VARCHAR2(50),
Issue_date DATE,
Return_date DATE
);

CREATE TABLE Library_Audit (
Audit_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
Book_id NUMBER(10),
Book_name VARCHAR2(100),
Author VARCHAR2(60),
Status VARCHAR2(20),
Issued_to VARCHAR2(50),
Issue_date DATE,
Return_date DATE,
Action VARCHAR2(10),
Changed_on DATE DEFAULT SYSDATE
);

-- Sample data
INSERT INTO Library VALUES (1001, 'DBMS Concepts', 'Ramez Elmasri', 'Available', NULL, NULL, NULL);
INSERT INTO Library VALUES (1002, 'OS Principles', 'Silberschatz', 'Issued', 'Alice', SYSDATE-5, SYSDATE+10);
COMMIT;
```

BEFORE UPDATE Trigger

```
CREATE OR REPLACE TRIGGER trg_library_before_update
BEFORE UPDATE ON Library
FOR EACH ROW
BEGIN
INSERT INTO Library_Audit (
Book_id, Book_name, Author, Status,
Issued_to, Issue_date, Return_date, Action, Changed_on
) VALUES (
:OLD.Book_id, :OLD.Book_name, :OLD.Author, :OLD.Status,
:OLD.Issued_to, :OLD.Issue_date, :OLD.Return_date, 'UPDATE', SYSDATE
);
```

```
END;
/
```

BEFORE DELETE Trigger

```
CREATE OR REPLACE TRIGGER trg_library_before_delete
BEFORE DELETE ON Library
FOR EACH ROW
BEGIN
INSERT INTO Library_Audit (
Book_id, Book_name, Author, Status,
Issued_to, Issue_date, Return_date, Action, Changed_on
) VALUES (
:OLD.Book_id, :OLD.Book_name, :OLD.Author, :OLD.Status,
:OLD.Issued_to, :OLD.Issue_date, :OLD.Return_date, 'DELETE', SYSDATE
);
END;
/
```

AFTER INSERT Trigger (Audit new insertions)

```
CREATE OR REPLACE TRIGGER trg_library_after_insert
AFTER INSERT ON Library
FOR EACH ROW
BEGIN
INSERT INTO Library_Audit (
Book_id, Book_name, Author, Status,
Issued_to, Issue_date, Return_date, Action, Changed_on
) VALUES (
:NEW.Book_id, :NEW.Book_name, :NEW.Author, :NEW.Status,
:NEW.Issued_to, :NEW.Issue_date, :NEW.Return_date, 'INSERT', SYSDATE
);
END;
/
```

Test the Triggers

```
-- Test UPDATE trigger
UPDATE Library SET Status = 'Issued', Issued_to = 'Bob' WHERE Book_id = 1001;
COMMIT;

-- Test DELETE trigger
DELETE FROM Library WHERE Book_id = 1002;
COMMIT;

-- View audit log
SELECT * FROM Library_Audit;
```

Problem Statement 09 — MongoDB CRUD Operations

Q1. Create database Employee.

```
use Employee
```

Q2. Create collection emp1 using createCollection method.

```
db.createCollection("emp1")
```

Q3. Insert 4-5 documents into emp1 collection.

```
db.emp1.insertMany([
{ eno: 1, ename: "Nilesh", address: "Nashik", sal: 12000 },
{ eno: 2, ename: "Priya", address: "Pune", sal: 8000 },
{ eno: 3, ename: "Rahul", address: "Mumbai", sal: 25000 },
{ eno: 4, ename: "Sneha", address: "Nagpur", sal: 3500 },
{ eno: 5, ename: "Manish", address: "Satara", sal: 18000 }
])
```

Q4. Display all documents.

```
db.emp1.find().pretty()
```

Q5. Insert document with custom object id.

```
db.emp1.insertOne({
_id: ObjectId("507f1f77bcf86cd799439011"),
eno: 6, ename: "Aditya", address: "Kolhapur", sal: 15000
})
```

Q6. Display all employees with salary > 5000.

```
db.emp1.find({ sal: { $gt: 5000 } }).pretty()
```

Q7. Display all employees with salary < 15000.

```
db.emp1.find({ sal: { $lt: 15000 } }).pretty()
```

Q8. Display employees with salary > 10000 and < 20000.

```
db.emp1.find({ sal: { $gt: 10000, $lt: 20000 } }).pretty()
```

Q9. Update salary of all employees by 10%.

```
db.emp1.updateMany(
{},
[{ $set: { sal: { $multiply: ["$sal", 1.10] } } }]
)
```

Q10. Delete employees with salary < 5000.

```
db.emp1.deleteMany({ sal: { $lt: 5000 } })
```

Q11. Rename collection emp1 to employee1.

```
db.emp1.renameCollection("employee1")
```

Q12. Display employees whose name starts with 'n' (case-insensitive).

```
db.employee1.find({ ename: { $regex: /^n/i } }).pretty()
```

Q13. Sort names in ascending order.

```
db.employee1.find({}, { ename: 1, _id: 0 }).sort({ ename: 1 })
```

Problem Statement 10 — MongoDB Aggregation & Indexing

Collection Setup

```
use Library

db.createCollection("Books")
db.createCollection("Products")

db.Books.insertMany([
{ book_id: 1, book_title: "DBMS", author_name: "Elmasri", publish_year: 2018 },
{ book_id: 2, book_title: "OS Concepts", author_name: "Silberschatz",publish_year: 2020 },
{ book_id: 3, book_title: "Algorithms", author_name: "Cormen", publish_year: 2009 },
{ book_id: 4, book_title: "Networking", author_name: "Elmasri", publish_year: 2016 }
])

db.Products.insertMany([
{ product_id: 1, name: "Laptop", price: 55000 },
{ product_id: 2, name: "Mouse", price: 800 },
{ product_id: 3, name: "Monitor", price: 12000 },
{ product_id: 4, name: "Keyboard",price: 1500 }
])
```

Q1. Create an index on the price field in Products.

```
db.Products.createIndex({ price: 1 })
```

Q2. Create a compound index on author_name and publish_year in Books.

```
db.Books.createIndex({ author_name: 1, publish_year: -1 })
```

Q3. View all indexes in Books and Products.

```
db.Books.getIndexes()
db.Products.getIndexes()
```

Q4. Drop the price index from Products.

```
db.Products.dropIndex({ price: 1 })
```

Q5. Find total number of books published by each author.

```
db.Books.aggregate([
{ $group: { _id: "$author_name", total_books: { $sum: 1 } } },
{ $sort: { total_books: -1 } }
])
```

Q6. Calculate average price of all products.

```
db.Products.aggregate([
{ $group: { _id: null, avg_price: { $avg: "$price" } } }
])
```

Q7. Find maximum and minimum price in Products.

```
db.Products.aggregate([
{ $group: {
```

```
_id: null,
max_price: { $max: "$price" },
min_price: { $min: "$price" }
}}
])
```

Q8. Count books published after 2015.

```
db.Books.aggregate([
{ $match: { publish_year: { $gt: 2015 } } },
{ $count: "books_after_2015" }
])
```

Q9. Display only book_title and author_name using $project.

```
db.Books.aggregate([
{ $project: { _id: 0, book_title: 1, author_name: 1 } }
])
```

Q10. Sort books collection by publish_year descending.

```
db.Books.aggregate([
{ $sort: { publish_year: -1 } }
])
```

Problem Statement 11 — MongoDB Queries (Restaurants)

Setup

```
use Restaurants

db.createCollection("REST1")

db.REST1.insertMany([
{ restaurant_id: 3001, name: "Spice Garden",
address: { building: "12", street: "MG Road", city: "Pune", zipcode: "411001" },
cuisine_type: "Indian", score: 85 },
{ restaurant_id: 3002, name: "The Burger Barn",
address: { building: "45", street: "FC Road", city: "Pune", zipcode: "411004" },
cuisine_type: "American", score: 92 },
{ restaurant_id: 3003, name: "Noodle House",
address: { building: "7", street: "Camp Road", city: "Pune", zipcode: "411001" },
cuisine_type: "Chinese", score: 78 },
{ restaurant_id: 3004, name: "Curry Palace",
address: { building: "23", street: "Baner Road", city: "Pune", zipcode: "411045" },
cuisine_type: "Indian", score: 88 }
])
```

Q1. Display all documents in the REST1 collection.

```
db.REST1.find().pretty()
```

Q2. Display restaurant_id, name, city, zipcode (exclude _id).

```
db.REST1.find(
{},
{ _id: 0, restaurant_id: 1, name: 1,
"address.city": 1, "address.zipcode": 1 }
)
```

Q3. Find restaurants with score > 80 and < 100.

```
db.REST1.find({ score: { $gt: 80, $lt: 100 } },
{ name: 1, score: 1, _id: 0 })
```

Q4. Update score of restaurant_id 3001 to 90.

```
db.REST1.updateOne(
{ restaurant_id: 3001 },
{ $set: { score: 90 } }
)
```

Q5. Update cuisine type of all Indian restaurants to 'Indian_Zatka'.

```
db.REST1.updateMany(
{ cuisine_type: "Indian" },
{ $set: { cuisine_type: "Indian_Zatka" } }
)
```

Q6. Delete the restaurant with restaurant_id 3003.

```
db.REST1.deleteOne({ restaurant_id: 3003 })
```

Q7. Delete all restaurants with score < 85.

```
db.REST1.deleteMany({ score: { $lt: 85 } })
```

Problem Statement 12 — MongoDB Queries (Students)

Setup

```
use School

db.createCollection("Students")

db.Students.insertMany([
{ student_id: 1, name: "Alice", age: 20, gender: "Female",
marks: { math: 92, science: 88, english: 85 } },
{ student_id: 2, name: "Bob", age: 19, gender: "Male",
marks: { math: 65, science: 70, english: 78 } },
{ student_id: 3, name: "Carol", age: 21, gender: "Female",
marks: { math: 95, science: 91, english: 82 } },
{ student_id: 4, name: "Dave", age: 22, gender: "Male",
marks: { math: 78, science: 84, english: 89 } },
{ student_id: 5, name: "Eve", age: 20, gender: "Female",
marks: { math: 55, science: 60, english: 72 } }
])
```

Q1. Find all students who are Male.

```
db.Students.find({ gender: "Male" }).pretty()
```

Q2. Find students who scored more than 90 in math.

```
db.Students.find({ "marks.math": { $gt: 90 } })
```

Q3. Find students aged 20 and above.

```
db.Students.find({ age: { $gte: 20 } })
```

Q4. Find students whose English marks are between 80 and 90.

```
db.Students.find({ "marks.english": { $gte: 80, $lte: 90 } })
```

Q5. Count the number of female students.

```
db.Students.countDocuments({ gender: "Female" })
```

Q6. Find the student with the highest science score.

```
db.Students.find({}, { name: 1, "marks.science": 1, _id: 0 })
.sort({ "marks.science": -1 })
.limit(1)
```

Q7. Update Bob's math score to 75.

```
db.Students.updateOne(
{ name: "Bob" },
{ $set: { "marks.math": 75 } }
)
```

Q8. Delete all students who scored below 70 in math.

```
db.Students.deleteMany({ "marks.math": { $lt: 70 } })
```

Q9. Find average marks in science for all students.

```
db.Students.aggregate([
```

```
{ $group: { _id: null, avg_science: { $avg: "$marks.science" } } }
])
```

Q10. Find number of students with marks > 80 in any subject.

```
db.Students.countDocuments({
$or: [
{ "marks.math": { $gt: 80 } },
{ "marks.science": { $gt: 80 } },
{ "marks.english": { $gt: 80 } }
]
})
```